

Source Code

Script 1: reddit_collection.py

```
"""
reddit_collection.py
-----
Collects Reddit comments from Pushshift monthly archive dumps for six K-
pop
comebacks during their release windows.

Background
    The original methodology (documented in the Interim Progress Report,
    March 2026) planned to use the Reddit API via PRAW. After the Reddit
    API access request was not approved in time, the project pivoted to
    the publicly available Pushshift monthly archive dumps, distributed
    as .zst-compressed newline-delimited JSON via Academic Torrents.

    This script replaces the earlier collection script (lost from local
    storage before it was committed to git). It reproduces the same
    methodology that was used to generate the raw CSVs already saved
    under data/raw/reddit/ and is included in the code appendix as the
    canonical reference for the Reddit collection step.

Methodology
    For each of the six target comebacks, Reddit comments are pulled
    from nine K-pop-related subreddits within a 14-day window starting
    on the release date (day 0 through day +13 inclusive).

    Filtering rules per comment:
        1. Subreddit must be in the target list.
        2. created_utc must fall within the comeback's 14-day window.
        3. body must not be empty, '[deleted]', or '[removed]'.
        4. If the comment is posted in the comeback group's *own*
           subreddit (e.g., r/Aespa for the Whiplash comeback), it is
           kept without a keyword filter. Activity in a group's home
           subreddit during its release window is assumed to be
           dominated by that comeback.
        5. Otherwise, the comment body must contain either the group
           name or the song title (case-insensitive substring match).

    Duplicate comment IDs across dump files are removed.

Output
    One CSV per comeback, written to data/raw/reddit/, with columns:
        id, subreddit, text, score, created_utc, comeback

Input
    Pushshift monthly comment dumps named 'RC_YYYY-MM.zst' placed in
    the directory referenced by DUMP_DIR below.

Usage
```

```

python reddit_collection.py

Dependencies
    pip install zstandard
"""

from __future__ import annotations

import csv
import io
import json
from datetime import datetime, timedelta
from pathlib import Path

import zstandard as zstd

# -----
# Configuration
# -----

# Comebacks under study. Release dates confirmed against each group's
# official release; keyword terms are lowercase substrings matched against
# the comment body.

COMEBACKS = [
    {
        "label": "StrayKids_ChkChkBoom",
        "group": "stray kids",
        "song": "chk chk boom",
        "release_date": datetime(2024, 7, 19),
        "own_sub": "straykids",
    },
    {
        "label": "aespa_Whiplash",
        "group": "aespa",
        "song": "whiplash",
        "release_date": datetime(2024, 10, 21),
        "own_sub": "Aespa",
    },
    {
        "label": "NCTDREAM_WhenImWithYou",
        "group": "nct dream",
        "song": "when i'm with you",
        "release_date": datetime(2024, 11, 11),
        "own_sub": "NCTDream",
    },
    {
        "label": "ATEEZ_IceOnMyTeeth",
        "group": "ateez",
        "song": "ice on my teeth",
        "release_date": datetime(2024, 11, 15),
    },

```

```

        "own_sub":      "ATEEZ",
    },
    {
        "label":        "TWICE_Strategy",
        "group":         "twice",
        "song":          "strategy",
        "release_date":  datetime(2024, 12, 6),
        "own_sub":       "twice",
    },
    {
        "label":        "IVE_RebelHeart",
        "group":         "ive",
        "song":          "rebel heart",
        "release_date":  datetime(2025, 1, 13),
        "own_sub":       "IVE",
    },
]

# 14-day window: [release_date, release_date + WINDOW_DAYS)
WINDOW_DAYS = 14

# Subreddits searched for every comeback. Comparisons are case-
insensitive.
SUBREDDITS = {
    "kpop", "kpopthoughts", "unpopularkpopopinions",
    "straykids", "aespa", "NCTDream", "ATEEZ", "twice", "IVE",
}
SUBREDDITS_LC = {s.lower() for s in SUBREDDITS}

# Paths. Absolute Windows paths so the script runs from any working
directory.
# The r"" prefix makes these raw strings so backslashes are treated
literally.
DUMP_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\comments")
OUTPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# CSV schema (order matters: matches existing raw files).
OUTPUT_FIELDS = ["id", "subreddit", "text", "score", "created_utc",
"comeback"]

# -----
# Pushshift .zst streaming
# -----

def read_zst_ndjson(path: Path):
    """
    Yield one JSON object per line from a zstd-compressed ndjson dump.
    The large max_window_size is required because Pushshift dumps are

```

```

compressed with long-range mode.
"""
dctx = zstd.ZstdDecompressor(max_window_size=2 ** 31)
with open(path, "rb") as f:
    with dctx.stream_reader(f) as reader:
        text_stream = io.TextIOWrapper(reader, encoding="utf-8")
        for line in text_stream:
            line = line.strip()
            if not line:
                continue
            try:
                yield json.loads(line)
            except json.JSONDecodeError:
                # Skip the occasional malformed line rather than
                # aborting the whole dump.
                continue

# -----
---
# Filtering
# -----
---

def keep_comment(obj: dict, comeback: dict) -> bool:
    """Apply subreddit, date-window, body, and keyword filters."""
    sub = obj.get("subreddit", "") or ""
    if sub.lower() not in SUBREDDITS_LC:
        return False

    # Date window
    try:
        ts = int(obj.get("created_utc", 0))
    except (ValueError, TypeError):
        return False
    comment_date = datetime.utcfromtimestamp(ts)
    start = comeback["release_date"]
    end = start + timedelta(days=WINDOW_DAYS)
    if not (start <= comment_date < end):
        return False

    # Body validity
    body = obj.get("body") or ""
    body_l = body.strip().lower()
    if not body_l or body_l in ("[deleted]", "[removed]"):
        return False

    # In the group's own subreddit, keep every valid comment in-window.
    if sub.lower() == comeback["own_sub"].lower():
        return True

    # Elsewhere, require a keyword match.
    return comeback["group"] in body_l or comeback["song"] in body_l

```

```

# -----
---
# Collection loop
# -----
---

def months_covering(start: datetime, end: datetime):
    """Yield (year, month) pairs covering the date range inclusively."""
    y, m = start.year, start.month
    while (y, m) <= (end.year, end.month):
        yield y, m
        if m == 12:
            y, m = y + 1, 1
        else:
            m += 1

def collect_for_comeback(comeback: dict) -> list[dict]:
    """Return the filtered, de-duplicated comments for one comeback."""
    start = comeback["release_date"]
    end = start + timedelta(days=WINDOW_DAYS)
    rows = []
    seen = set()

    for year, month in months_covering(start, end):
        dump = DUMP_DIR / f"RC_{year}-{month:02d}.zst"
        if not dump.exists():
            print(f" [warn] Missing dump for {year}-{month:02d}: {dump}")
            continue

        print(f" Reading {dump.name} ...")
        for obj in read_zst_ndjson(dump):
            if not keep_comment(obj, comeback):
                continue
            cid = obj.get("id")
            if not cid or cid in seen:
                continue
            seen.add(cid)
            rows.append({
                "id": cid,
                "subreddit": obj.get("subreddit", ""),
                "text": obj.get("body", ""),
                "score": obj.get("score", 0),
                "created_utc": obj.get("created_utc", 0),
                "comeback": comeback["label"],
            })

    return rows

def main() -> None:
    for cb in COMEBACKS:
        print(f"\n>>> Collecting {cb['label']} ...")

```

```

        rows = collect_for_comeback(cb)
        out_path = OUTPUT_DIR / f"{cb['label']}_reddit_raw.csv"
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=OUTPUT_FIELDS)
            writer.writeheader()
            writer.writerows(rows)
        print(f"  Saved {len(rows)} comments -> {out_path}")

if __name__ == "__main__":
    main()

```

Script 2: reddit_filter.py

```

"""
reddit_filter.py
-----
Post-hoc keyword filter applied to the six raw Reddit CSVs produced by
reddit_collection.py. Replaces the initial case-insensitive substring
match with a stricter, word-boundary-aware ruleset that eliminates
false positives caused by group names colliding with common English
words (notably IVE vs. the contraction "I've", and TWICE vs. the
adverb "twice").

```

Rationale

The original collection script used ``group_name in text.lower()``. This worked well for distinctive tokens (aespa, ATEEZ, stray kids, nct dream) but introduced substantial noise for IVE and TWICE. Audit of the raw IVE data showed that comments pulled from non-IVE subreddits matched "ive" as a substring of common words (I've, received, alive, give) in the large majority of cases.

This filter is applied symmetrically to all six comebacks so the refinement does not bias one group relative to the others.

Filter rules (applied only to comments outside the comeback's own sub)
A comment is kept if any one of:

1. The song title phrase matches (case-insensitive). If the title is itself a common English word or phrase (Strategy, When I'm With You), the comment must also contain the group name token or a recognised member name.
2. The group name pattern matches. For all-caps stylized names (IVE, TWICE), matching is case-sensitive to distinguish "IVE" from "I've" / "ive".
3. A member name (word-boundary, case-insensitive) appears.

Own-sub comments are always kept. Topical relevance is implicit when a user posts in the group's own subreddit.

Input

Six raw CSVs in 01_raw_data/reddit/

Output

Six cleaned CSVs in 01_raw_data/reddit/cleaned/
Plus a _filter_summary.csv table of rows kept vs. dropped.

"""

```
from __future__ import annotations
```

```
import re
```

```
import pandas as pd
```

```
from pathlib import Path
```

```
# -----  
---  
# Configuration  
# -----  
---
```

```
INPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-  
analysis\01_raw_data\reddit")
```

```
OUTPUT_DIR = INPUT_DIR / "cleaned"
```

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
# Per-comeback filter specification.
```

```
# own_sub:          subreddit name (case-insensitive comparison)
```

```
# song_pattern:     regex for the song title
```

```
# song_ambiguous:   True if the song title is a common English word/phrase  
#                   and therefore requires additional context to match
```

```
# group_pattern:    regex for the group name
```

```
# group_case:       True = case-sensitive match (for all-caps  
stylizations)
```

```
# members:          list of member names (word-boundary, case-insensitive)
```

```
FILTERS = {
```

```
    "IVE_RebelHeart": {
```

```
        "file":          "IVE_RebelHeart_reddit_raw.csv",
```

```
        "own_sub":       "ive",
```

```
        "song_pattern":  r"rebel\s+heart",
```

```
        "song_ambiguous": False,
```

```
        "group_pattern": r"\bIVE\b",
```

```
        "group_case":    True,
```

```
        "members":       ["yujin", "gaeul", "rei", "wonyoung", "liz",
```

```
"leeseo"],
```

```
    },
```

```
    "TWICE_Strategy": {
```

```
        "file":          "TWICE_Strategy_reddit_raw.csv",
```

```
        "own_sub":       "twice",
```

```
        "song_pattern":  r"\bstrategy\b",
```

```
        "song_ambiguous": True,
```

```
        "group_pattern": r"\bTWICE\b",
```

```
        "group_case":    True,
```

```
        "members":       ["nayeon", "jeongyeon", "momo", "sana", "jihyo",  
                           "mina", "dahyun", "chaeyoung", "tzuyu"],
```

```
    },
```

```
    "aespa_Whiplash": {
```

```

        "file": "aespa_Whiplash_reddit_raw.csv",
        "own_sub": "aespa",
        "song_pattern": r"\bwhiplash\b",
        "song_ambiguous": False,
        "group_pattern": r"\baespa\b",
        "group_case": False,
        "members": ["karina", "giselle", "winter", "ningning"],
    },
    "StrayKids_ChkChkBoom": {
        "file": "StrayKids_ChkChkBoom_reddit_raw.csv",
        "own_sub": "straykids",
        "song_pattern": r"chk\s*chk\s*boom",
        "song_ambiguous": False,
        "group_pattern": r"\bstray\s*kids\b|\bSKZ\b",
        "group_case": False,
        "members": ["bang chan", "bangchan", "lee know", "leeknow",
                    "changbin", "hyunjin", "felix", "seungmin",
                    "han jisung"],
    },
    "NCTDREAM_WhenImWithYou": {
        "file": "NCTDREAM_WhenImWithYou_reddit_raw.csv",
        "own_sub": "nctdream",
        "song_pattern": r"when\s+i'?m\s+with\s+you",
        "song_ambiguous": True,
        "group_pattern": r"\bnct\s*dream\b",
        "group_case": False,
        "members": ["renjun", "jeno", "haechan", "jaemin",
                    "chenle", "jisung"],
    },
    "ATEEZ_IceOnMyTeeth": {
        "file": "ATEEZ_IceOnMyTeeth_reddit_raw.csv",
        "own_sub": "ateez",
        "song_pattern": r"ice\s+on\s+my\s+teeth",
        "song_ambiguous": False,
        "group_pattern": r"\bateez\b",
        "group_case": False,
        "members": ["hongjoong", "seonghwa", "yunho", "yeosang",
                    "wooyoung", "jongho", "mingi"],
    },
}

# -----
# Matching logic
# -----

def match_group(text: str, spec: dict) -> bool:
    flags = 0 if spec["group_case"] else re.IGNORECASE
    return bool(re.search(spec["group_pattern"], text, flags))

def match_song(text: str, spec: dict) -> bool:

```

```

    return bool(re.search(spec["song_pattern"], text, re.IGNORECASE))

def match_member(text: str, spec: dict) -> bool:
    for m in spec["members"]:
        if re.search(rf"\b{re.escape(m)}\b", text, re.IGNORECASE):
            return True
    return False

def is_true_mention(text: str, spec: dict) -> bool:
    if not isinstance(text, str) or not text:
        return False

    if match_song(text, spec):
        if spec["song_ambiguous"]:
            if match_group(text, spec) or match_member(text, spec):
                return True
        else:
            return True

    if match_group(text, spec):
        return True

    if match_member(text, spec):
        return True

    return False

# -----
---
# Main
# -----
---

def filter_file(label: str, spec: dict) -> dict:
    path = INPUT_DIR / spec["file"]
    df = pd.read_csv(path)
    total = len(df)

    own_mask = df["subreddit"].str.lower() == spec["own_sub"].lower()
    other_df = df[~own_mask].copy()
    kept_other = other_df["text"].apply(lambda t: is_true_mention(t,
spec))

    cleaned = pd.concat([df[own_mask], other_df[kept_other]],
ignore_index=True)

    out_path = OUTPUT_DIR / spec["file"].replace("_raw.csv",
"_cleaned.csv")
    cleaned.to_csv(out_path, index=False)

    return {

```

```

        "comeback":      label,
        "total_raw":     total,
        "own_sub_kept":  int(own_mask.sum()),
        "other_raw":     int((~own_mask).sum()),
        "other_kept":    int(kept_other.sum()),
        "total_clean":   len(cleaned),
        "dropped":       total - len(cleaned),
        "drop_pct":      round(100 * (total - len(cleaned)) / total, 1),
        "output":        out_path.name,
    }

def main() -> None:
    summary = [filter_file(label, spec) for label, spec in
FILTERS.items()]
    summary_df = pd.DataFrame(summary)

    print("\n=== FILTER SUMMARY ===")
    print(summary_df.to_string(index=False))

    summary_path = OUTPUT_DIR / "_filter_summary.csv"
    summary_df.to_csv(summary_path, index=False)
    print(f"\nSummary written to {summary_path}")

if __name__ == "__main__":
    main()

```

Script 3: language_filter.py

```

"""
language_filter.py
-----
Filters the cleaned Reddit CSVs (and YouTube CSVs) down to English-only
comments for downstream preprocessing, manual labelling, and
classification.

Rationale
    The Interim Progress Report (March 2026) committed the methodology to
    English-language content only. K-pop fan comments across both
platforms
    regularly contain Korean (in Hangul and in romanised form),
Indonesian,
    Spanish, Thai, Turkish, and other languages. Including these in the
    corpus would produce garbage features for an English-trained
classifier.

    The `langdetect` library is the standard Python tool for this task but
    is known to be unreliable on short text (it reports high confidence on
    obviously-wrong guesses like "Wow." -> Polish, "Rebel Heart slaps"
    -> Dutch). Because fan comments are overwhelmingly short, a naive

```

langdetect filter would discard a substantial amount of genuine English content.

Filter logic

1. Non-Latin script check: if a comment contains substantial Hangul, CJK, Cyrillic, Thai, Arabic, or Hebrew characters, it is flagged as non-English and excluded.
2. Very short comments (under 15 words) are kept by default. The small amount of non-English noise this admits is preferable to the high rate of false-negative English exclusions langdetect produces on short text.
3. For comments of 15 words or more, langdetect is consulted. Only comments detected as English with confidence ≥ 0.80 are kept.

This favours recall (keeping genuine English) over precision (excluding every non-English comment), which is the right trade-off for a small training corpus.

Input

Cleaned CSVs from 01_raw_data/reddit/cleaned/
YouTube raw CSVs from their existing location (adjust YT_INPUT_DIR)

Output

English-only CSVs in 01_raw_data/reddit/cleaned_en/
A summary table reporting rows kept vs. dropped per file.

"""

```
from __future__ import annotations

import re
import pandas as pd
from pathlib import Path
from langdetect import detect_langs, DetectorFactory, LangDetectException

# -----
# Configuration
# -----

# Deterministic langdetect behaviour across runs
DetectorFactory.seed = 0

REDDIT_INPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned")
REDDIT_OUTPUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned_en")
REDDIT_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# YouTube data is stored as a single combined CSV rather than per-comeback
# files. If YT_INPUT_FILE is set to None, YouTube filtering is skipped.
```

```

YT_INPUT_FILE = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_raw.csv")
YT_OUTPUT_FILE = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_en.csv")
YT_TEXT_COL = "comment"
YT_GROUP_COL = "group_comeback"

LONG_TEXT_WORDS_THRESHOLD = 15
LANGDETECT_CONFIDENCE = 0.80

# Unicode ranges for non-Latin scripts common in K-pop fan comments
NON_LATIN_RANGES = [
    (0xAC00, 0xD7AF), # Hangul syllables
    (0x1100, 0x11FF), # Hangul Jamo
    (0x3130, 0x318F), # Hangul compat
    (0x3040, 0x309F), # Hiragana
    (0x30A0, 0x30FF), # Katakana
    (0x4E00, 0x9FFF), # CJK unified ideographs
    (0x0400, 0x04FF), # Cyrillic
    (0x0E00, 0x0E7F), # Thai
    (0x0600, 0x06FF), # Arabic
    (0x0590, 0x05FF), # Hebrew
]

# -----
---
# Filtering logic
# -----
---

def non_latin_char_ratio(text: str) -> float:
    """Return the fraction of characters that fall in non-Latin
    scripts."""
    if not text:
        return 0.0
    non_latin = 0
    total = 0
    for ch in text:
        cp = ord(ch)
        if ch.isspace() or not ch.isalpha():
            continue
        total += 1
        for lo, hi in NON_LATIN_RANGES:
            if lo <= cp <= hi:
                non_latin += 1
                break
    if total == 0:
        return 0.0
    return non_latin / total

def is_english(text: str) -> bool:
    """Decide whether a comment should be kept as English content."""

```

```

if not isinstance(text, str) or not text.strip():
    return False

# Rule 1: heavy non-Latin script -> drop
if non_latin_char_ratio(text) >= 0.20:
    return False

# Rule 2: short text -> keep (langdetect unreliable)
word_count = len(text.split())
if word_count < LONG_TEXT_WORDS_THRESHOLD:
    return True

# Rule 3: longer text -> consult langdetect with confidence threshold
try:
    langs = detect_langs(text)
    top = langs[0]
    return top.lang == "en" and top.prob >= LANGDETECT_CONFIDENCE
except LangDetectException:
    # langdetect gave up (e.g. pure emoji/symbol content). Default to
    # keeping, since the non-Latin check already ran.
    return True

# -----
---
# File processing
# -----
---

def filter_csv(in_path: Path, out_path: Path, text_col: str = "text") ->
dict:
    df = pd.read_csv(in_path)
    total = len(df)
    if text_col not in df.columns:
        raise ValueError(f"{in_path.name}: no '{text_col}' column")

    mask = df[text_col].apply(is_english)
    kept = df[mask].copy()
    kept.to_csv(out_path, index=False)

    return {
        "file":    in_path.name,
        "total":   total,
        "kept":    int(mask.sum()),
        "dropped": total - int(mask.sum()),
        "drop_pct": round(100 * (total - int(mask.sum())) / total, 1)
                    if total else 0.0,
        "output":  out_path.name,
    }

def filter_youtube_single_file() -> list:
    """
    Filter the combined YouTube CSV and report per-comeback drop rates.

```

```

Returns a list of summary dicts (one per comeback + one total row).
"""
df = pd.read_csv(YT_INPUT_FILE)
if YT_TEXT_COL not in df.columns:
    raise ValueError(f"{YT_INPUT_FILE.name}: no '{YT_TEXT_COL}'
column")

print(f"Filtering YouTube: {YT_INPUT_FILE.name} ...")
mask = df[YT_TEXT_COL].apply(is_english)
kept = df[mask].copy()

YT_OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)
kept.to_csv(YT_OUTPUT_FILE, index=False)

rows = []
for cb, sub in df.groupby(YT_GROUP_COL):
    sub_mask = sub[YT_TEXT_COL].apply(is_english)
    total = len(sub)
    k = int(sub_mask.sum())
    rows.append({
        "file":      f"youtube:{cb}",
        "total":     total,
        "kept":      k,
        "dropped":   total - k,
        "drop_pct":  round(100 * (total - k) / total, 1) if total else
0.0,
        "output":    YT_OUTPUT_FILE.name,
    })
# Combined row
rows.append({
    "file":      "youtube:TOTAL",
    "total":     len(df),
    "kept":      int(mask.sum()),
    "dropped":   len(df) - int(mask.sum()),
    "drop_pct":  round(100 * (len(df) - int(mask.sum())) / len(df), 1)
if len(df) else 0.0,
    "output":    YT_OUTPUT_FILE.name,
})
return rows

def main() -> None:
    summary = []

    # Reddit cleaned CSVs
    for in_path in sorted(REDDIT_INPUT_DIR.glob("*_cleaned.csv")):
        out_path = REDDIT_OUTPUT_DIR /
in_path.name.replace("_cleaned.csv", "_en.csv")
        print(f"Filtering Reddit: {in_path.name} ...")
        summary.append(filter_csv(in_path, out_path, text_col="text"))

    # YouTube combined CSV
    if YT_INPUT_FILE is not None and YT_INPUT_FILE.exists():
        summary.extend(filter_youtube_single_file())

```

```

    else:
        print(f"YouTube file not found at {YT_INPUT_FILE}; skipping
        YouTube.")

    summary_df = pd.DataFrame(summary)
    print("\n=== LANGUAGE FILTER SUMMARY ===")
    print(summary_df.to_string(index=False))

    summary_path = REDDIT_OUTPUT_DIR / "_language_filter_summary.csv"
    summary_df.to_csv(summary_path, index=False)
    print(f"\nSummary written to {summary_path}")

if __name__ == "__main__":
    main()

```

Script 4: preprocess.py

```

"""
preprocess.py
-----
Preprocesses the labelled YouTube and Reddit corpora and the full
English-filtered corpora for WEKA sentiment classification.

Steps applied to every comment:
    1. Encoding normalisation (fixes "™ and similar UTF-8 mojibake
       common in Pushshift Reddit dumps)
    2. Lowercase
    3. URL removal
    4. Emoji and non-ASCII removal
    5. Number removal (digits stripped)
    6. Punctuation removal
    7. Tokenisation (whitespace split after cleaning)
    8. Stopword removal (NLTK English stopwords)
    9. Stemming (Porter stemmer, matching standard NLP pipeline)
    10. Rejoin to cleaned string

This pipeline matches the existing cleaned_comment column in
youtube_comments_for_labelling.csv, with the addition of encoding
normalisation and explicit stopwords/stemming steps for consistency
with the Interim Report's stated methodology (C2).

Outputs
-----
02_processed_data/
    youtube_labelled_processed.csv - 300 labelled YouTube rows
    reddit_labelled_processed.csv - 297 labelled Reddit rows
    combined_labelled_processed.csv - merged for WEKA training
    youtube_full_processed.csv - full YouTube corpus (2619 rows)
    reddit_full_processed.csv - full Reddit corpus (~15k rows)

04_weka_files/

```

```

training_data.arff          - combined labelled set for WEKA
youtube_full.arff          - full YouTube corpus for WEKA
reddit_full.arff           - full Reddit corpus for WEKA
"""

from __future__ import annotations

import re
import pandas as pd
from pathlib import Path

import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download required NLTK data if not already present
nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)

# -----
# Paths
# -----

BASE = Path(r"C:\Users\lenovo\kpop-sentiment-analysis")

YOUTUBE_LABELLED = BASE / "03_labeled_data" /
"youtube_comments_for_labelling.csv"
REDDIT_LABELLED = BASE / "03_labeled_data" /
"reddit_labelling_template.csv"
YOUTUBE_FULL = BASE / "01_raw_data" / "youtube" /
"youtube_comments_en.csv"
REDDIT_EN_DIR = BASE / "01_raw_data" / "reddit" / "cleaned_en"

OUT_PROCESSED = BASE / "02_processed_data"
OUT_WEKA = BASE / "04_weka_files"
OUT_PROCESSED.mkdir(parents=True, exist_ok=True)
OUT_WEKA.mkdir(parents=True, exist_ok=True)

# -----
# Cleaning pipeline
# -----

STOP_WORDS = set(stopwords.words('english'))
STEMMER = PorterStemmer()

# Encoding fix map for common UTF-8 mojibake sequences from Pushshift
ENCODING_FIXES = {
    'â€™': "'",
    'âœ': "'",
    'â€': "'",

```

```

    'â€': '-',
    'â€': '-',
    'Â': '',
    'â€|': '...',
}

def fix_encoding(text: str) -> str:
    """Fix common UTF-8 mojibake sequences from Pushshift dumps."""
    for bad, good in ENCODING_FIXES.items():
        text = text.replace(bad, good)
    return text

def clean(text: str) -> str:
    """Apply the full cleaning pipeline to a single comment."""
    if not isinstance(text, str) or not text.strip():
        return ''

    # Step 1: encoding fix
    text = fix_encoding(text)

    # Step 2: lowercase
    text = text.lower()

    # Step 3: remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Step 4: remove non-ASCII (emoji, Hangul residue, etc.)
    text = text.encode('ascii', errors='ignore').decode('ascii')

    # Step 5: remove digits
    text = re.sub(r'\d+', '', text)

    # Step 6: remove punctuation (keep spaces)
    text = re.sub(r'^\w\s]', '', text)

    # Step 7: tokenise
    tokens = text.split()

    # Step 8: stopword removal
    tokens = [t for t in tokens if t not in STOP_WORDS]

    # Step 9: stemming
    tokens = [STEMMER.stem(t) for t in tokens]

    # Step 10: rejoin
    return ' '.join(tokens)

# -----
# ARFF export

```

```

# -----
---

def to_arff(df: pd.DataFrame,
            text_col: str,
            label_col: str,
            relation_name: str,
            path: Path,
            label_values: list[str] | None = None) -> None:
    """
    Write a simple bag-of-words ARFF file suitable for WEKA
    StringToWordVector.

    The text column is written as a STRING attribute so that WEKA's
    StringToWordVector filter can convert it to TF-IDF features before
    classification. This is the standard approach for text classification
    in WEKA when you don't want to pre-compute the vocabulary yourself.
    """
    if label_values is None:
        label_values = sorted(df[label_col].dropna().unique().tolist())

    lines = [
        f"@relation {relation_name}",
        "",
        "@attribute text STRING",
        f"@attribute class {{{','.join(label_values)}}}",
        "",
        "@data",
    ]

    for _, row in df.iterrows():
        text = str(row[text_col]).replace('"', "\\").replace("'", "\\'")
        label = str(row[label_col])
        # ARFF string values are wrapped in single quotes
        lines.append(f"'{text}',{label}")

    path.write_text('\n'.join(lines), encoding='utf-8')
    print(f" -> ARFF: {path} ({len(df)} instances)")

# -----
---

# Processing functions
# -----
---

def process_youtube_labelled() -> pd.DataFrame:
    """Load, filter to labelled rows, apply cleaning, return DataFrame.

    Note: the existing 'cleaned_comment' column in the source file used a
    lighter pipeline (no stopwords removal, no stemming). This function
    re-cleans from the raw 'comment' column using the full pipeline
    defined
    above, which matches the methodology stated in the Interim Progress

```

```

Report (C2: tokenisation, stopwords removal, stemming via NLTK).
The 'cleaned_comment' column is intentionally ignored.
"""

df = pd.read_csv(YOUTUBE_LABELLED)
# Keep only labelled rows
df = df[df['sentiment'].notna()].copy()
df = df.rename(columns={'sentiment': 'label', 'comment': 'raw_text',
                        'group_comeback': 'comeback'})
# Re-clean from raw text (not cleaned_comment)
df['cleaned_text'] = df['raw_text'].apply(clean)
df['source'] = 'youtube'
return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]

def process_reddit_labelled() -> pd.DataFrame:
    """Load labelled Reddit rows, apply cleaning, return DataFrame."""
    df = pd.read_csv(REDDIT_LABELLED)
    df = df[df['label'].notna()].copy()
    df = df.rename(columns={'text': 'raw_text'})
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['source'] = 'reddit'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]

def process_youtube_full() -> pd.DataFrame:
    """Load the full English-filtered YouTube corpus and clean it."""
    df = pd.read_csv(YOUTUBE_FULL)
    df = df.rename(columns={'comment': 'raw_text',
                            'group_comeback': 'comeback'})
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['label'] = '?' # unknown - to be predicted by classifier
    df['source'] = 'youtube'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]

def process_reddit_full() -> pd.DataFrame:
    """Load all six English-filtered Reddit CSVs and clean them."""
    frames = []
    for path in sorted(REDDIT_EN_DIR.glob("*_en.csv")):
        sub = pd.read_csv(path)
        sub = sub.rename(columns={'text': 'raw_text'})
        frames.append(sub)
    df = pd.concat(frames, ignore_index=True)
    df['cleaned_text'] = df['raw_text'].apply(clean)
    df['label'] = '?'
    df['source'] = 'reddit'
    return df[['comeback', 'raw_text', 'cleaned_text', 'label', 'source']]

# -----
# ---
# Main
# -----
# ---

```

```

def main() -> None:
    label_values = ['negative', 'neutral', 'positive']

    print("Processing YouTube labelled ...")
    yt_lab = process_youtube_labelled()
    yt_lab.to_csv(OUT_PROCESSED / "youtube_labelled_processed.csv",
index=False)
    print(f"    {len(yt_lab)} rows |
{yt_lab['label'].value_counts().to_dict()}")

    print("Processing Reddit labelled ...")
    rd_lab = process_reddit_labelled()
    rd_lab.to_csv(OUT_PROCESSED / "reddit_labelled_processed.csv",
index=False)
    print(f"    {len(rd_lab)} rows |
{rd_lab['label'].value_counts().to_dict()}")

    print("Combining labelled sets ...")
    combined = pd.concat([yt_lab, rd_lab], ignore_index=True)
    combined.to_csv(OUT_PROCESSED / "combined_labelled_processed.csv",
index=False)
    print(f"    {len(combined)} rows |
{combined['label'].value_counts().to_dict()}")

    print("Processing full YouTube corpus ...")
    yt_full = process_youtube_full()
    yt_full.to_csv(OUT_PROCESSED / "youtube_full_processed.csv",
index=False)
    print(f"    {len(yt_full)} rows")

    print("Processing full Reddit corpus ...")
    rd_full = process_reddit_full()
    rd_full.to_csv(OUT_PROCESSED / "reddit_full_processed.csv",
index=False)
    print(f"    {len(rd_full)} rows")

    print("\nExporting ARFF files ...")
    to_arff(combined, 'cleaned_text', 'label',
            'kpop_sentiment_training', OUT_WEKA / 'training_data.arff',
            label_values)
    to_arff(yt_full, 'cleaned_text', 'label',
            'youtube_full', OUT_WEKA / 'youtube_full.arff',
            label_values)
    to_arff(rd_full, 'cleaned_text', 'label',
            'reddit_full', OUT_WEKA / 'reddit_full.arff',
            label_values)

    print("\nDone. Summary:")
    print(f"    Training instances : {len(combined)}")
    print(f"    YouTube to predict : {len(yt_full)}")
    print(f"    Reddit to predict   : {len(rd_full)}")
    print(f"    ARFF files in       : {OUT_WEKA}")

```

```
if __name__ == "__main__":
    main()
```

Script 5: sample_for_labelling.py

```
"""
sample_for_labelling.py
-----

Draws a stratified random sample of 50 comments per comeback from both
Reddit and YouTube English-filtered corpora, producing two labelling
workbooks. Adds an empty 'label' column for the user to fill with
positive, negative, or neutral.

Rationale
    Random stratified sampling is reproducible (fixed seed) and avoids
    self-selection bias that would occur if the labeller chose which
    comments to label. This is the standard practice for building a
    labelled training set from a larger corpus.

Output
    03_labeled_data/reddit_labelling_template.csv
    03_labeled_data/youtube_labelling_template.csv
"""

from __future__ import annotations

import pandas as pd
from pathlib import Path

RANDOM_SEED = 42
SAMPLE_PER_COMEBACK = 50

REDDIT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\reddit\cleaned_en")
YT_FILE = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_comments_en.csv")
OUT_DIR = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\03_labeled_data")
OUT_DIR.mkdir(parents=True, exist_ok=True)

def sample_reddit() -> pd.DataFrame:
    frames = []
    for path in sorted(REDDIT_DIR.glob("*_en.csv")):
        df = pd.read_csv(path)
        if len(df) < SAMPLE_PER_COMEBACK:
            print(f" WARN: {path.name} has only {len(df)} rows, taking
all")
            sampled = df.copy()
        else:
```

```

        sampled = df.sample(n=SAMPLE_PER_COMEBACK,
random_state=RANDOM_SEED)
        frames.append(sampled)
        result = pd.concat(frames, ignore_index=True)
        result["label"] = "" # empty column for manual labelling
        result["source"] = "reddit"
        return result

def sample_youtube() -> pd.DataFrame:
    df = pd.read_csv(YT_FILE)
    frames = []
    for cb, sub in df.groupby("group_comeback"):
        if len(sub) < SAMPLE_PER_COMEBACK:
            print(f" WARN: {cb} has only {len(sub)} rows, taking all")
            sampled = sub.copy()
        else:
            sampled = sub.sample(n=SAMPLE_PER_COMEBACK,
random_state=RANDOM_SEED)
            frames.append(sampled)
        result = pd.concat(frames, ignore_index=True)
        result["label"] = ""
        result["source"] = "youtube"
        return result

def main() -> None:
    print("Sampling Reddit ...")
    rd = sample_reddit()
    rd_out = OUT_DIR / "reddit_labelling_template.csv"
    rd.to_csv(rd_out, index=False)
    print(f" -> {rd_out} ({len(rd)} rows)")

    print("Sampling YouTube ...")
    yt = sample_youtube()
    yt_out = OUT_DIR / "youtube_labelling_template.csv"
    yt.to_csv(yt_out, index=False)
    print(f" -> {yt_out} ({len(yt)} rows)")

if __name__ == "__main__":
    main()

```

Script 6: collect_youtube_metrics.py

```

"""
collect_youtube_metrics.py
-----
Collects video-level engagement metrics for the six comeback MVs
using the YouTube Data API v3.

Saves to: 01_raw_data/youtube/youtube_metrics.csv

```

```

"""

import requests
import pandas as pd
from pathlib import Path

# Your YouTube Data API v3 key
# Replace with your actual key from credentials folder
API_KEY = "YOUR_YOUTUBE_API_KEY_HERE"

# Video IDs identified from the comments corpus
VIDEOS = {
    "aespa_Whiplash": "jWQx2f-CERU",
    "ATEEZ_IceOnMyTeeth": "5Of10lcHLb8",
    "NCTDREAM_WhenImWithYou": "B1qq8IvzSz4",
    "StrayKids_ChkChkBoom": "0P0aQreFs8w",
    "TWICE_Strategy": "Sz_wWzgh-vQ",
    "IVE_RebelHeart": "g36q0ZLvYgQ",
}

OUTPUT = Path(r"C:\Users\lenovo\kpop-sentiment-
analysis\01_raw_data\youtube\youtube_metrics.csv")

def get_video_metrics(video_id: str, api_key: str) -> dict:
    url = "https://www.googleapis.com/youtube/v3/videos"
    params = {
        "part": "statistics,snippet",
        "id": video_id,
        "key": api_key,
    }
    response = requests.get(url, params=params)
    data = response.json()

    if not data.get("items"):
        print(f" WARNING: No data returned for {video_id}")
        return {}

    item = data["items"][0]
    stats = item["statistics"]

    return {
        "video_id": video_id,
        "view_count": int(stats.get("viewCount", 0)),
        "like_count": int(stats.get("likeCount", 0)),
        "comment_count": int(stats.get("commentCount", 0)),
        "title": item["snippet"]["title"],
        "published_at": item["snippet"]["publishedAt"],
    }

def main():
    rows = []
    for comeback, video_id in VIDEOS.items():

```

```

    print(f"Fetching metrics for {comeback} ({video_id}) ...")
    metrics = get_video_metrics(video_id, API_KEY)
    if metrics:
        metrics["comeback"] = comeback
        # Compute like-to-view ratio
        if metrics["view_count"] > 0:
            metrics["like_to_view_ratio"] = metrics["like_count"] /
metrics["view_count"]
        else:
            metrics["like_to_view_ratio"] = 0.0
        rows.append(metrics)
        print(f"  Views: {metrics['view_count']:,}  Likes:
{metrics['like_count']:,}  Comments: {metrics['comment_count']:,}")

    df = pd.DataFrame(rows)
    cols = ["comeback", "video_id", "title", "published_at",
            "view_count", "like_count", "comment_count",
"like_to_view_ratio"]
    df = df[cols]

    OUTPUT.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUTPUT, index=False)
    print(f"\nSaved to {OUTPUT}")
    print(df[["comeback", "view_count", "like_count", "comment_count",
"like_to_view_ratio"]].to_string(index=False))

if __name__ == "__main__":
    main()

```